

Name-Free Combinators for the Rho Calculus

A repair, a linear encoding, and the one operation that was missing

F1R3FLY.io research note

September 2026 — draft 3 (rewritten)

Abstract

The name-free concurrent combinators of *Name-free combinators for concurrency* were proposed as a target into which the rho calculus embeds with no binders at all. This note asks whether the embedding is correct. It is not, and the repairs are instructive.

The published calculus is unsound, and the defect is the conflation of the rho calculus drop with the guard that consumes a message and runs its payload: under $*@P = P$ that conflation makes every running process a redex. Separating the two constructors repairs it and leaves two sound presentations, both given here as operational theories — one keeping the equation and decoding by it, one omitting it, sorting message payloads as processes, and decoding by the ϵ rule, which is then literally the drop clause of Meredith–Radestock substitution promoted from the metatheory. Neither can be assembled piecemeal from the other, and only the second lets a decoder return an optimised representative rather than an equal one, which is what an implementation that normalises on decode actually does.

Repaired, the combinators still fall short of the rho calculus by exactly one capability: they have a name destructor and no name constructor, so they can open a name that arrives at runtime but never build one. A name-closure lemma makes this the bottom edge of an expressiveness lattice over subsets of the atoms, developed here in place of a single separation theorem. The lattice carries two orderings, expressive and observational, which do not coincide, because the calculus has no name hiding; both are drawn. Four constructors close the gap, one for each node a rebuilt name can have, and the translation then covers the whole calculus with no fragment restriction.

Reflection also buys an unexpected efficiency. A continuation can be quoted and released by a three-atom gate rather than rewritten atom by atom, so prefix elimination is linear where Yoshida’s is $\Theta(3^d)$ in the nesting depth of inputs. Correctness is established as full abstraction relative to encoded contexts, in the sense of the companion note, and the restriction to encoded observers is shown to be forced — which is also where the storage combinator earns its place, since it holds a continuation invisibly rather than as an observable message. A final section gives an alternative presentation in which the message and the store are not primitive but the nullary members of the constructor family, leaving a calculus of routers, one destructor, and constructors.

Contents

1	Introduction	2
2	The rho calculus	4

3	The rho combinators	5
3.1	Notation	5
3.2	Formation	5
3.3	Equations, and the two presentations	5
3.4	The defect	7
3.5	The storage combinator	7
3.6	The name constructors	7
4	Where the decoding happens	9
5	Context-labelled bisimilarity	9
6	The translation	10
6.1	Why two phases	10
6.2	Phase one: into the combinators with a binder	10
6.3	Phase two: eliminating the binder	12
6.4	The permutation discipline	13
6.5	Freshness	13
7	Correctness	14
8	Complexity of the target term	16
9	An expressiveness lattice	17
9.1	Three invariants	17
9.2	The constructors: a Boolean sublattice	18
9.3	The destructor	19
9.4	Duplication	19
9.5	Routers, storage, and the observer class	19
9.6	Summary	20
10	An alternative presentation: constructors only	21
11	Corrigenda to [5]	22
12	Open questions	22

1 Introduction

The rho calculus¹ has no name-restriction operator: names are quoted processes, and freshness is obtained by quotation rather than by a binder. It nonetheless has a binder, the input prefix. Yoshida showed how to eliminate the input binder of the asynchronous π -calculus in favour of combinators, at the cost of keeping **new**. The combination is suggestive: a calculus with no **new**, compiled by a discipline that removes the remaining binder, should give a concurrency

¹The name is an acronym and a pun. It abbreviates the **reflective higher order** calculus; dropping the periods from r.h.o. gives the English transliteration of the Greek letter following π , marking the calculus as successor to the π -calculus. It is written “rho calculus”, never with the Greek letter.

calculus with no bound names whatsoever. That is what [5] proposed. This note asks whether the proposal works.

Findings.

1. **One equation breaks the calculus.** The published combinators inherit $*@P \equiv P$ from a presentation in which quote and drop are mutually inverse up to structural congruence. In the combinator setting the drop is not a process former but a guard, and since every name is $@Q$ for some Q , the identification makes an arbitrary running process match the guard. Proposition 3.5: for all P, Q , $Q \mid m(@Q, @P) \longrightarrow P$. The repair is to separate the drop from the guard, which leaves the equation optional; §3.3 gives both presentations and §4 compares them.
2. **Reflection was half-implemented.** The rule $e(a) \mid m(a, @P) \longrightarrow P$ opens a name that arrives at runtime. Nothing builds one. Lemma 9.1: combinator reduction never produces a name that was not already present in the redex, hereditarily. Rho reduction does, at every step, because substituting under a quotation turns $@(*y \mid R)$ into $@(Q \mid R)$ — a name in neither participant. Lemma 9.1 is the bottom edge of the lattice of §9: no combinator term of the published calculus is equivalent to $\text{for}(y \leftarrow a) @(*y \mid *y)!(\mathbf{0})$. The lattice has two orderings, expressive and observational, which do not coincide, because the calculus has no name hiding.
3. **Constructors close the gap.** The missing operation is the construction of a name from names, supplied by constructors of the shape $\text{cons}_i(a, b, c) \mid m(a, @P) \mid m(b, @Q) \longrightarrow m(c, @(P \mid Q))$, one for each node a rebuilt name can have. Four suffice when apparatus names are fixed at compile time (Proposition 3.12); the allocation of §6.3 needs one per atom shape (Remark 3.13). Either way there is no name-opaque fragment to restrict to.
4. **The storage combinator is about opacity, not power.** $q(a, P)$, released by a forwarder rather than by a message, satisfies the no-name-creation lemma and so adds nothing to expressiveness. What it adds is silence: it is a message that only a redirection can read, which is what stored code and intermediate results of name construction need if the encoding is to be opaque to observers (§3.5, Remark 7.12).
5. **Prefix elimination is linear.** Reflection allows a continuation to be quoted once and released by a three-atom gate. Yoshida must instead push the prefix through the body, splitting at every parallel composition and delaying every atom, which costs a constant factor per level of nesting: Proposition 8.2 computes $(3^{d-1} + 1)/2$ atoms on an explicit family where the reflective encoding gives $3d + 1$.
6. **Correctness is full abstraction relative to encoded contexts.** Following [4], observation is by context-labelled bisimilarity with the observer class taken to be the image of the translation on source contexts. Theorem 7.9; and Proposition 7.10, that the restriction is forced rather than convenient, by the same argument that forces it there.

§10 gives the alternative presentation in which m and q are the nullary constructors. §11 lists the corrigenda to [5].

Notation. [5] writes $*$ for three distinct things: Yoshida’s replication, the rho calculus drop, and the combinator that consumes a message and runs its payload. We write $\mathbf{e}$ for the last and reserve $*x$ for the drop. The distinction between the second and the third is one of the points of §4.

2 The rho calculus

We follow [3].

Definition 2.1 (Syntax).

$$\begin{aligned} P, Q &::= \mathbf{0} \mid \text{for}(y \leftarrow x)P \mid x!(P) \mid *x \mid P \mid Q \\ x, y &::= @P \end{aligned}$$

$\text{fn}(P)$ is as usual, with $\text{fn}(\text{for}(y \leftarrow x)P) = \{x\} \cup (\text{fn}(P) \setminus \{y\})$. $\mathcal{N}^*(P)$ is the set of names occurring anywhere in P , including hereditarily inside quotations.

Definition 2.2 (Structural congruence, name equivalence). $\equiv$ is the least congruence containing α -equivalence and making $(\text{Proc}, |, \mathbf{0})$ an abelian monoid. $\equiv_{\mathcal{N}}$ is the least congruence on names with $@*x \equiv_{\mathcal{N}} x$, and $@P \equiv_{\mathcal{N}} @Q$ whenever $P \equiv Q$.

The converse identification $*@P \equiv P$ is deliberately absent. Its work is done by substitution.

Definition 2.3 (Semantic substitution). On names, $y\{@Q/y\} = @Q$ and $@P\{@Q/y\} = @(P\{@Q/y\})$. On processes, the structural clauses together with

$$(*y)\{@Q/y\} = Q, \quad (*z)\{@Q/y\} = *z \quad (z \neq y).$$

The drop clause is the whole of the difference between semantic and syntactic substitution. Syntactic substitution returns $*@Q$, and one then needs $*@Q \equiv Q$ to recover the meaning. Semantic substitution decodes at the moment of substitution and returns a process bisimilar to any other faithful decoding rather than congruent to a canonical one; the decoder is free to return an optimised representative.

Definition 2.4 (Reduction).

$$\frac{x_t \equiv_{\mathcal{N}} x_s}{\text{for}(y \leftarrow x_t)P \mid x_s!(Q) \longrightarrow P\{@Q/y\}} \quad \frac{P \longrightarrow P'}{P \mid Q \longrightarrow P' \mid Q} \quad \frac{P \equiv P' \quad P' \longrightarrow Q' \quad Q' \equiv Q}{P \longrightarrow Q}$$

Definition 2.5. P is *guarded-drop* if every $*x$ in P has x a variable bound by an enclosing input.

Lemma 2.6. *Guarded-drop is preserved by reduction, and no subterm $*@Q$ ever arises from a guarded-drop term.*

Proof. A drop can acquire a ground name only by substitution, and Definition 2.3 eliminates the drop rather than instantiating it. Every other clause is structural. $\square$

A term $*@Q$ may still be written by hand, and is then inert. For such terms $*@Q$ and Q are related by the decoding bisimulation rather than by $\equiv$, which is the latitude the original presentation was designed to leave. Source terms are assumed guarded-drop throughout.

3 The rho combinators

3.1 Notation

We present the combinators as an operational theory: formation judgements, equations, and named rewrite rules, each under a context assigning sorts to its variables. There are two sorts, $\mathbf{N}$ for names and $\mathbf{P}$ for processes. This is heavier than the grammar of [5] and earns its weight in §3.3, where two presentations differ precisely in which sort a message payload has, a distinction a single-sorted grammar hides.

3.2 Formation

$$\begin{array}{ll}
a, b : \mathbf{N} \vdash \mathbf{m}(a, b) : \mathbf{P} & a, b : \mathbf{N} \vdash \mathbf{b}_l(a, b) : \mathbf{P} \\
a, b, c : \mathbf{N} \vdash \mathbf{d}(a, b, c) : \mathbf{P} & a, b : \mathbf{N} \vdash \mathbf{b}_r(a, b) : \mathbf{P} \\
a : \mathbf{N} \vdash \mathbf{k}(a) : \mathbf{P} & a, b, c : \mathbf{N} \vdash \mathbf{s}(a, b, c) : \mathbf{P} \\
a, b : \mathbf{N} \vdash \mathbf{fw}(a, b) : \mathbf{P} & a : \mathbf{N} \vdash \mathbf{e}(a) : \mathbf{P} \\
a : \mathbf{N}, p : \mathbf{P} \vdash \mathbf{q}(a, p) : \mathbf{P} & a, b, c : \mathbf{N} \vdash \mathbf{cons}(a, b, c) : \mathbf{P} \\
a, b, c : \mathbf{N} \vdash \mathbf{cons}_m(a, b, c) : \mathbf{P} & a, b, c, f : \mathbf{N} \vdash \mathbf{cons}_d(a, b, c, f) : \mathbf{P} \\
a, b, c, f : \mathbf{N} \vdash \mathbf{cons}_s(a, b, c, f) : \mathbf{P} & \vdash \mathbf{0} : \mathbf{P} \\
p, q : \mathbf{P} \vdash p \mid q : \mathbf{P} & p : \mathbf{P} \vdash @p : \mathbf{N} \\
a : \mathbf{N} \vdash *a : \mathbf{P} &
\end{array}$$

We write $\mathbf{cons}$ for the parallel constructor, $\mathbf{cons}_l$ where the subscript aids reading. Note that $*$ — and $\mathbf{e}$ are distinct constructors: the first is the drop of the rho calculus, the second the guard that consumes a message and runs its payload. Conflating them is the defect of §3.4.

3.3 Equations, and the two presentations

Definition 3.1 (Equations). Common to both presentations:

$$\begin{array}{l}
p, q : \mathbf{P} \vdash p \mid q = q \mid p \\
p, q, r : \mathbf{P} \vdash (p \mid q) \mid r = p \mid (q \mid r) \\
p : \mathbf{P} \vdash p \mid \mathbf{0} = p \\
a : \mathbf{N} \vdash @*a = a
\end{array}$$

The two presentations differ in one equation and, consequently, in the sort of a message payload and in the form of the $\mathbf{eval}$ rule.

Presentation A (identified). Add

$$p : \mathbf{P} \vdash *@p = p$$

so that $@$ — and $*$ — are inverse bijections between the sorts. Message payloads are names, $\mathbf{m}(a, b)$ with $b : \mathbf{N}$, and decoding is performed by the equation:

$$a, v : \mathbf{N} \vdash \mathbf{eval} : \mathbf{e}(a) \mid \mathbf{m}(a, v) \longrightarrow *v$$

which is the intended process because $*@p = p$.

Presentation B (original). Omit that equation. Message payloads are then processes, $m(a, p)$ with $p : P$, matching the output constructor $x!(P)$ of the source calculus, and decoding is performed by the rule:

$$a : N, p : P \vdash \text{eval} : e(a) \mid m(a, p) \longrightarrow p$$

Remark 3.2 (The presentations are not interchangeable piecemeal). One cannot omit $*@p = p$ and keep name-sorted payloads. With name-sorted payloads the only available rule is $e(a) \mid m(a, v) \longrightarrow *v$, and without the equation the result is a stuck drop; while pattern-matching the payload as $@p$ is ambiguous, since $@*a = a$ lets any v be read as $@*v$, so the rule could return either $*v$ or, when v is literally $@Q$, the process Q — and without the equation these differ. Sorting the payload as a process removes the ambiguity, because the payload is then given rather than recovered. The two presentations are thus each internally forced, and the choice between them is the choice of where decoding happens.

Definition 3.3 (Rules). Common to both presentations, with the payload metavariable read as a name in A and as a process in B:

$$\begin{aligned} a, b, c, v : N &\vdash \text{dist} : d(a, b, c) \mid m(a, v) \longrightarrow m(b, v) \mid m(c, v) \\ a, v : N &\vdash \text{kill} : k(a) \mid m(a, v) \longrightarrow \mathbf{0} \\ a, b, v : N &\vdash \text{forward} : \text{fw}(a, b) \mid m(a, v) \longrightarrow m(b, v) \\ a, b, v : N &\vdash \text{bind}_l : b_l(a, b) \mid m(a, v) \longrightarrow \text{fw}(v, b) \\ a, b, v : N &\vdash \text{bind}_r : b_r(a, b) \mid m(a, v) \longrightarrow \text{fw}(b, v) \\ a, b, c, v : N &\vdash \text{synch} : s(a, b, c) \mid m(a, v) \longrightarrow \text{fw}(b, c) \\ a, b : N, p : P &\vdash \text{quote} : \text{fw}(a, b) \mid q(a, p) \longrightarrow m(b, @p) \\ a, b, c : N, p, r : P &\vdash \text{make}_l : \text{cons}_l(a, b, c) \mid m(a, @p) \mid m(b, @r) \longrightarrow m(c, @(p \mid r)) \\ &\text{and } \text{make}_A \text{ for each atom shape } A, \text{ as in Definition 3.8} \end{aligned}$$

together with the presentation's own *eval*, closed under parallel composition and under the equations, with subjects matched up to $\equiv_{\mathcal{N}}$.

For the remainder we write payloads in presentation A's notation, $m(a, @p)$; under B read $m(a, p)$. No rule, lemma or proof below distinguishes the two.

Reduction is nondeterministic and is meant to be. Two consumers may compete for one message, as in $d(a, b, c) \mid k(a) \mid m(a, v)$; and two producers may compete for one consumer, as in $\text{fw}(a, b) \mid m(a, v) \mid q(a, p)$, where *fw* is the consumer at a and *m* and *q* both offer a value there. Neither is pathological and neither is new: they are the two sides of the race the communication rule has always had.

Remark 3.4 (Departures from the published rules). Three corrections are folded into Definition 3.3 and are worth flagging, since they are easy to reintroduce. The binders must produce forwarders, $b_r(a, b) \mid m(a, v) \longrightarrow \text{fw}(b, v)$ and $b_l(a, b) \mid m(a, v) \longrightarrow \text{fw}(v, b)$; a version producing a message merely duplicates *forward* and loses the only function these atoms have, which is to bind the received name into a redirection — the links of kinds (o1) and (o2) in §6 depend on it. *fw* is binary throughout. And in the constructors the delivery channel f must appear in the atom's own argument list, distinct from the input channels.

3.4 The defect

Proposition 3.5. *Suppose $*-$ and e are the same constructor, as in [5], where the guard is written $*$; and suppose $*@p = p$. Then for all P, Q ,*

$$Q \mid m(@Q, @P) \longrightarrow P.$$

Proof. Every name is $@Q$ for some Q , so under the conflation $*(@Q) = Q$. Reduction is closed under the equations, so the redex $*(a) \mid m(a, @P)$ with $a = @Q$ matches $Q \mid m(@Q, @P)$, and the rule fires. $\square$

An arbitrary running process can therefore be annihilated and replaced by an arbitrary payload, by anyone who knows its quotation. No faithfulness theorem survives this.

Remark 3.6. The hypothesis that does the damage is the conflation, not the equation. The published grammar lists the guard under the process production alongside parallel composition, which invites reading it as the drop; separating the two constructors, as the formation rules above do, repairs the calculus and leaves both presentations of §3.3 available. It is worth being explicit that Proposition 3.5 is therefore not an argument against $*@p = p$; the argument about that equation is in §4, and it is not an argument from soundness.

3.5 The storage combinator

Remark 3.7 (q is a message only a forwarder can read). The rule **quote** has the same right-hand side as **forward**. So $q(a, p)$ and $m(a, @p)$ are indistinguishable to a forwarder, and differ everywhere else: a message at a may be consumed by any of d, k, b_r, b_l, s, e , a constructor or **fw**, whereas $q(a, p)$ may be consumed only by **fw**. Since $fw(a, a) \mid q(a, p) \longrightarrow m(a, @p)$, q is not a message awaiting a trigger but the introduction form for a name constant, of which m is the already-manifest case.

q adds no expressive power — it satisfies Lemma 9.1 — and is not needed for the translation to be definable. What it provides is opacity; see Remark 7.12.

3.6 The name constructors

Every rule but the constructors moves names around or opens one. The constructors build one.

Definition 3.8 (The constructor family). For each atom shape A of the signature, of arity n over names, there is a constructor of arity $n + 1$:

$$\begin{aligned} a_1, \dots, a_n, f, v_1, \dots, v_n : \mathbf{N} \vdash \text{make}_A : \\ \text{cons}_A(a_1, \dots, a_n, f) \mid m(a_1, v_1) \mid \dots \mid m(a_n, v_n) \longrightarrow m(f, @A(v_1, \dots, v_n)) \end{aligned}$$

together with $\text{cons}_\parallel$ for parallel composition, taking quotations to the quotation of their composite, and the constant $@0$. For an atom shape with a process argument the member is stated directly in presentation B , where payloads are process-sorted; the case of q is

$$a, b, c : \mathbf{N}, p_1, p_2 : \mathbf{P} \vdash \text{make}_q : \text{cons}_q(a, b, c) \mid m(a, p_1) \mid m(b, p_2) \longrightarrow m(c, q(@p_1, p_2))$$

and cons_m follows the same pattern. The family is finite because the signature is, and it includes a member for each constructor, so the calculus is closed under its own constructions.

Remark 3.9 (Process arguments and the two presentations). Shapes with a process argument are where the presentations of §3.3 come apart a third time. Under B the member above is direct: $@p_1 : N$ and $p_2 : P$ are exactly the sorts q wants. Under A, whose payloads are names, the same member must be written $@q(v_1, *v_2)$, coercing a name back to a process by $*-$ — which denotes the intended atom only because $*@p = p$. So A can build these constructors, but only by leaning on its equation, and B builds them without it. This bears on §6.3, which erects a q at allocated names in Example 6.9.

Remark 3.10. Each constructor takes names and returns a name: it assembles the quotation of a single node from the quotations of its children, and the assembled name is delivered as an ordinary message, so it may be used as a subject, as a payload, or as input to a further constructor. Chaining them builds a name of any shape. The operation performed is substitution of a received name into a static term under a quotation, which is what the rho communication rule does and what [5] has no counterpart for.

Remark 3.11 (Machinery at a computed name). Constructors deliver names as messages, but a name is often wanted as a *subject*. The two steps

$$\text{cons}_A(a_1, \dots, a_n, f) \mid m(a_1, \nu_1) \mid \dots \longrightarrow m(f, @A(\vec{\nu})), \quad e(f) \mid m(f, @A(\vec{\nu})) \longrightarrow A(\vec{\nu})$$

erect a live atom whose subject was computed at run time. This is the construction on which §6.3 depends, and it is worth stating separately because it is easy to believe, wrongly, that the combinators can only forward to a received name and never build machinery at one.

Proposition 3.12 (Static-apparatus compilation). *If the apparatus names of the translation are fixed at compile time, four members of the family suffice: cons_l , cons_m , cons_d and cons_s .*

Proof. A name to be rebuilt is $@[S]$ for a source term S with the bound name free in it, and $[-]$ is a homomorphism, so the nodes of $[S]$ are those of S . Take the path from an occurrence to the root; every node off the path is static and is supplied as a constant message, joined on by cons_l . On the path: an occurrence under $*-$ is the received name itself; a parallel composition needs cons_s ; an output has image $m([v], @[T])$ and needs cons_m ; and an input has image $D([v]; \vec{p}) \mid G \mid \vec{L}$ in which $[v]$ occurs only in the head atom, d or s . Duplication of the received name is supplied by d . $\square$

Remark 3.13 (Four is not enough for phase two). Proposition 3.12 is conditional on static apparatus, and §6.3 abandons that condition: allocated apparatus must be erected by Remark 3.11, which calls for a constructor at every atom shape occurring in compiled output. The full family of Definition 3.8 is therefore not a fastidious closure condition but a requirement of the translation. The optimisations of Remark 6.4 likewise put a bound name into an e or a fw argument, so they too cost constructors.

Remark 3.14 (A more exotic alternative). The family can be collapsed into one atom indexed by a static context: $\text{cons}[C](a, c) \mid m(a, v) \longrightarrow m(c, @C[v])$ for a combinator context C with holes in name positions. This is derivable from the family by chaining, and conversely each member is the instance $C = A(-, \dots, -)$. It shortens the translation at the price of an atom parameterised by a context rather than by names — a considerable conceptual cost, which we do not pay, though the two formulations have the same reach.

4 Where the decoding happens

Both presentations of §3.3 are sound, and every result in this note holds under either. They differ in where the decoding of a name into the process it quotes is performed, and that difference has consequences worth stating rather than settling by fiat.

A puts the decoding in the equations. $*@p = p$ makes $@-$ and $*-$ inverse bijections, so $N \cong P$ and the two sorts carry the same information. Decoding is then not an event: $*v$ is the process, and **eval** need only produce the term $*v$ and let the equation do the rest. The presentation is economical and the rule is uniform in the payload.

B puts the decoding in the rule. Without the equation, the quotation of a process is an opaque name until some rule opens it, and **eval** is that rule. This is the arrangement of [3], where the corresponding clause is the drop case of substitution, $(*y)\{ @Q/y \} = Q$. Compare it with the rule:

$$(*y)\{ @Q/y \} = Q \quad \text{against} \quad e(a) \mid m(a, p) \longrightarrow p.$$

These are the same equation, the first in the metatheory of a calculus with substitution and the second in a calculus without one.

What B affords. Because decoding is an event rather than an identity, the decoder is free to return any process bisimilar to the one quoted, rather than one equal to it. Under A that freedom is not available: $*@p$ and p are equal, so any two decodings of the same name are equal, and a decoder that returns something else is not implementing the equation.

This is not a hypothetical distinction. The F1R3FLY implementation normalises on decode: what is handed back has been rearranged, and bound variables have been replaced by de Bruijn indices. Neither transformation is derivable from the equations of Definition 3.1, which give only the monoid laws and $@*a = a$. Under A the implementation is therefore not sound for the stated theory; under B it is, because the theory only ever claims the decoded process up to the equivalence of §5. An implementation that normalises is evidence about which presentation is being implemented, and it is B.

The surviving equation is load-bearing in both. $@*a = a$ is what makes it possible to send a name at all. A message carries a process in B, so “send the name x ” is $m(v, *x)$, whose quotation is $@*x = x$; in A the payload is x directly and the equation is what identifies the two readings. Every name-forwarding clause of the translation depends on it.

A remark on sorts. Under A the two sorts are isomorphic, so one may ask what work the sorting does. It does the work of Proposition 3.5: an isomorphism is not an identification of constructors, and keeping e distinct from $*-$ is what prevents an arbitrary process from matching a guard. The published single-sorted grammar had no way to say this.

5 Context-labelled bisimilarity

We use the equivalence of the companion note [4] rather than barbs, for consistency with it. The two are close here — both calculi are name-passing, so barbs are an adequate observation

— but the context-labelled form states the restriction on observers as part of the relation rather than as a side condition on a set of names, and that restriction is the substance of Theorem 7.9.

Definition 5.1. For a calculus with reduction $\longrightarrow$ and contexts $\mathcal{C}$, write $P \xrightarrow{C} P'$ when $C[P] \longrightarrow P'$ and $P \xRightarrow{C} P'$ when $C[P] \longrightarrow^* P'$. For $\mathcal{D} \subseteq \mathcal{C}$, a symmetric $\mathcal{R}$ is an $\mathcal{D}$ -bisimulation if $P \mathcal{R} Q$ and $P \xrightarrow{C} P'$ with $C \in \mathcal{D}$ imply $Q \xRightarrow{C} Q'$ with $P' \mathcal{R} Q'$. Write $\approx_{\mathcal{D}}$ for the largest such.

On the source, $\mathcal{D} = \mathcal{C}_{\text{rho}}$, written $\approx_{\text{rho}}$. On the target, $\mathcal{D} = \llbracket \mathcal{C}_{\text{rho}} \rrbracket$, the encodings of source contexts. As in [4] the labels are enabling contexts, not minimal ones; minimality is the right long-run discipline and is developed in [1], and nothing below needs it. Without it the label set is larger and $\approx_{\mathcal{D}}$ correspondingly finer, which makes completeness harder rather than easier.

6 The translation

The translation is in two phases. The first compiles the rho calculus into the combinators extended with a name binder, as Yoshida compiles the π -calculus into combinators extended with `new`. The second eliminates the binder. Splitting the work this way is not a convenience: an earlier single-phase version of this note allocated apparatus names by syntactic path, and §6.1 exhibits the term on which that is unsound.

6.1 Why two phases

Consider $R = \text{for}(y \leftarrow a)y!(*y)$, whose occurrence of y is the combined output-subject and forwarded-name case, so the atom is deleted and two links share one proxy c ; and

$$W = \text{for}(z \leftarrow b)(*z \mid *z) \mid b!(R) \mid a!(X) \mid a!(Y).$$

In the source, the communication on b decodes both drops, giving $R \mid R$ with the copies alpha-distinct, so one takes $@X$ and the other $@Y$ and $W \longrightarrow^* @X!(X) \mid @Y!(Y)$.

Under a compilation whose apparatus names are functions of syntactic position, `e` instantiates the same compiled text at both drop sites, so both copies carry the same c . After the links fire the soup holds

$$\text{fw}(c, \llbracket @X \rrbracket) \mid \text{fw}(c, \llbracket @Y \rrbracket) \mid \text{m}(c, \llbracket @X \rrbracket) \mid \text{m}(c, \llbracket @Y \rrbracket)$$

and nothing pairs a forwarder with the message from its own copy: a run yields $\llbracket @X!(Y) \rrbracket \mid \llbracket @Y!(X) \rrbracket$, which an encoded observer listening at $\llbracket @X \rrbracket$ distinguishes from the source. The invariant that fails is the one separating *static occurrences* from *run-time instances*: distinct occurrences do get distinct names, and that is what a positional scheme establishes, but a single occurrence instantiated twice does not.

A binder repairs this at once, because two instances of $(\text{new } c)P$ are alpha-distinct. That is what phase one buys, and eliminating the binder afterwards is what phase two must do without giving it back.

6.2 Phase one: into the combinators with a binder

Let RC' be the calculus of §3 extended with $z : \mathbf{N} \vdash (\text{new } z)P : P$, with the usual scope-extrusion equation and α -conversion.

	occurrence of y	in the quoted body	link
(o1)	input subject $\text{for}(z \leftarrow y)S$	subject $\rightarrow c_i$	$\mathbf{b}_l(p_i, c_i)$
(o2)	output subject $y!(S)$	subject $\rightarrow c_i$	$\mathbf{b}_r(p_i, c_i)$
(o3)	forwarded name $v!(*y)$	atom deleted	$\mathbf{fw}(p_i, \llbracket v \rrbracket)$
(o4)	drop $*y$ in process position	$\rightarrow \mathbf{e}(c_i)$	$\mathbf{fw}(p_i, c_i)$
(o5)	inside a quotation $@S, y \in \text{fn}(S)$	subject $\rightarrow c_i$	constructor chain for S

Table 1: Occurrence kinds. In (o5) the link is the chain of Proposition 3.12 rebuilding $@[S]$ from the received name. When several kinds coincide in one atom, the atom is deleted and the links are emitted together.

Definition 6.1 (Gate and distributor). For b, c fresh, $G(p, S) := (\text{new } b, c) (\mathbf{s}(p, b, c) \mid \mathbf{q}(b, S) \mid \mathbf{e}(c))$; and $D(x; p_0) := \mathbf{0}$ with $p_0 := x$, while for $k \geq 1$

$$D(x; p_0, \dots, p_k) := (\text{new } q_1, \dots, q_{k-1}) (\mathbf{d}(x, p_0, q_1) \mid \mathbf{d}(q_1, p_1, q_2) \mid \dots \mid \mathbf{d}(q_{k-1}, p_{k-1}, p_k)).$$

Lemma 6.2. $G(p, S)$ has no reduction and is not a message. For any v ,

$$G(p, S) \mid \mathbf{m}(p, v) \longrightarrow \mathbf{fw}(b, c) \mid \mathbf{q}(b, S) \mid \mathbf{e}(c) \longrightarrow \mathbf{m}(c, @S) \mid \mathbf{e}(c) \longrightarrow S,$$

and these are the only reductions available. $D(x; \vec{p}) \mid \mathbf{m}(x, v) \longrightarrow^* \mathbf{m}(p_0, v) \mid \dots \mid \mathbf{m}(p_k, v)$ in k steps.

Proof. The names p, b, c are distinct, so no other pairing is possible, and $\mathbf{s}$ discards the payload. Before the forwarder is released there is no $\mathbf{fw}$ at b , so $\mathbf{q}(b, S)$ is inert by Remark 3.7. $\square$

Occurrences of a bound name y in a body P fall into five kinds, four at which y is used as a name and one at which a new name must be built.

Definition 6.3 (Phase one). On names, $\llbracket @P \rrbracket := @[P]$. On processes,

$$\begin{aligned} \llbracket \mathbf{0} \rrbracket &= \mathbf{0} \\ \llbracket P \mid Q \rrbracket &= \llbracket P \rrbracket \mid \llbracket Q \rrbracket \\ \llbracket x!(P) \rrbracket &= \mathbf{m}(\llbracket x \rrbracket, @[P]) \\ \llbracket \text{for}(y \leftarrow x)P \rrbracket &= (\text{new } p_0, \dots, p_k, c_1, \dots, c_k) (D(\llbracket x \rrbracket; \vec{p}) \mid G(p_0, B) \mid L_1 \mid \dots \mid L_k) \end{aligned}$$

where $o_1, \dots, o_k$ enumerate the free occurrences of y in P , each o_i has proxy c_i , L_i and the residue in B are given by Table 1, and B is $\llbracket P \rrbracket$ with every occurrence treated accordingly.

Remark 6.4. Three special cases are cheaper: $\llbracket \text{for}(y \leftarrow x)\mathbf{0} \rrbracket = \mathbf{k}(\llbracket x \rrbracket)$, $\llbracket \text{for}(y \leftarrow x)*y \rrbracket = \mathbf{e}(\llbracket x \rrbracket)$, and $\llbracket \text{for}(y \leftarrow x)v!(*y) \rrbracket = \mathbf{fw}(\llbracket x \rrbracket, \llbracket v \rrbracket)$ for $v \neq y$. By Remark 3.13 they are not free.

Example 6.5. The witness of §6.1 is repaired at this phase. Two drop sites instantiate $@[R]$, and the copies are alpha-distinct, so the soup is $\mathbf{fw}(c_1, v_X) \mid \mathbf{m}(c_1, v_X) \mid \mathbf{fw}(c_2, v_Y) \mid \mathbf{m}(c_2, v_Y)$ and the crossed redex does not exist.

6.3 Phase two: eliminating the binder

Phase two runs each instance at an *address*, a name it receives at run time, and allocates its bound names beneath that address. Two ingredients suffice; neither needs a rule the calculus does not have.

Definition 6.6 (Allocation). Fix $K := @m(@0, @0)$ and put $L\sigma := @m(\sigma, @0)$, $R\sigma := @m(\sigma, K)$. Then

$$\text{ALLOC}(t; t_L, t_R) := d(t, u_1, u_2) \mid \text{cons}_m(u_1, z_0, t_L) \mid m(z_0, @0) \mid \text{cons}_m(u_2, z_1, t_R) \mid m(z_1, K).$$

Lemma 6.7. $\text{ALLOC}(t; t_L, t_R) \mid m(t, \sigma) \longrightarrow^* m(t_L, L\sigma) \mid m(t_R, R\sigma)$ in three steps, and L, R are injective with disjoint images.

Proof. d delivers σ to u_1 and u_2 ; each cons_m then has both its inputs. Injectivity and disjointness hold because $m(\sigma, @0)$ and $m(\sigma, K)$ determine σ and differ in their second argument, $@0 \neq K$ modulo $\equiv_{\mathcal{N}}$. $\square$

Iterating gives a binary tree of addresses beneath σ ; we write $\sigma \cdot w$ for the leaf at path w . Distinct leaves are distinct, and if neither of σ, τ is a prefix of the other then their leaf sets are disjoint — the prefix-freedom condition of [4].

Definition 6.8 (Phase two). Let $\llbracket P \rrbracket$ have bound names $z_1, \dots, z_n$ under a single **new**, arranged by scope. Its phase-two image is the closed term

$$\llbracket P \rrbracket^\sharp := \text{ALLOC-tree}(r; t_1, \dots, t_n, t_{\text{next}}) \mid \text{ERECT}(t_1, \dots, t_n)$$

where r is a static channel at which the instance receives its address, ALLOC-tree is the iteration of Definition 6.6 producing $n + 1$ leaves, and ERECT contains, for each atom $A(z_{i_1}, \dots, z_{i_m})$ of $\llbracket P \rrbracket$, the pair

$$\text{cons}_A(t_{i_1}, \dots, t_{i_m}, f_A) \mid e(f_A)$$

of Remark 3.11, which erects that atom at the allocated names. Atoms with no bound name are emitted unchanged. The spare leaf t_{next} is posted at r for the next instance.

Example 6.9 (The recursion combinator, run). $D_x = \text{for}(y \leftarrow x)(x!(\ast y) \mid \ast y)$ has occurrences of kinds (o3) and (o4), so phase one gives seven bound names, which phase two allocates as $p_0, q_1, p_1, p_2, c, b, c'$ together with a spare σ' . With $v = @[\llbracket D_x \rrbracket]^\sharp$ and the address σ present at r :

$\begin{aligned} & \text{ALLOC-tree} \mid m(r, \sigma) \longrightarrow^* \dots \\ & \text{ERECT} \longrightarrow^* d(x, p_0, q_1) \mid d(q_1, p_1, p_2) \mid s(p_0, b, c') \\ & \quad \mid q(b, e(c)) \mid e(c') \mid fw(p_1, x) \mid fw(p_2, c) \mid e(c) \\ & \longrightarrow m(p_0, v) \mid m(q_1, v) \\ & \longrightarrow m(p_1, v) \mid m(p_2, v) \\ & \longrightarrow fw(b, c') \\ & \longrightarrow m(c', @e(c)) \\ & \longrightarrow e(c) \\ & \longrightarrow m(x, v) \\ & \longrightarrow m(c, v) \\ & \longrightarrow \llbracket D_x \rrbracket^\sharp \end{aligned}$	$\begin{aligned} & \text{eight leaves beneath } \sigma \\ & d(x, p_0, q_1) \mid m(x, v) \\ & d(q_1, p_1, p_2) \\ & s(p_0, b, c') \\ & \text{quote} \\ & e(c') \\ & fw(p_1, x) \quad \text{re-send} \\ & fw(p_2, c) \\ & e(c) \quad \text{next instance} \end{aligned}$
--	--

with $m(r, \sigma \cdot w_{\text{next}})$ posted for it. The residue is the initial configuration at a strictly deeper address, so no two unfoldings share a leaf.

6.4 The permutation discipline

Two live instances share every static channel of Definition 6.8 — r , the allocation gadget’s internal channels, the f_A — and may cross-match freely at them. Whether that costs anything depends on what crosses, and the distinction is sharp enough to be the design rule of phase two: *apparatus may be permuted, values may not*. Source values never reach a static channel, because after its first step each instance moves them only through allocated apparatus; that is why the failure of §6.1 cannot recur, since there X and Y themselves travelled through a shared proxy.

Apparatus is another matter, and the naive reading of “harmless” is false. A constructor with two address-valued inputs at static channels can draw one from each of two instances, erecting an atom whose subject belongs to one and whose continuations belong to another — after which values do cross. The condition that rules this out is worth isolating.

Definition 6.10 (Single-address-input discipline). A phase-two image satisfies the discipline if every constructor placed at a statically named channel has at most one input carrying an address, and every join of two or more addresses occurs at a channel derived from a single address.

Lemma 6.11 (Permutation). *Let two instances run at prefix-incomparable σ, τ under Definition 6.10. Then every reachable configuration is one in which each instance holds only leaves of a single address, and the reachable configurations differ from the uncrossed ones by a permutation of $\{\sigma, \tau\}$ among the instances.*

Proof. A single-input constructor at a static channel emits a name determined by the one address it consumed, so its output is a leaf of that address; the cross-match merely exchanges which instance’s chain consumes which address, which is the permutation. A multi-address join, by the discipline, sits at a channel derived from one address, and by Lemma 6.7 that channel is a leaf of that address alone, so only that instance’s chain can deliver to it. Hence no atom is erected from leaves of two addresses, and by prefix-incomparability the two leaf sets stay disjoint. $\square$

Remark 6.12 (One way to satisfy it). Erect each atom from a single copy of its instance’s address, threading it through single-input chains and performing every join at an address-derived channel, rather than allocating the leaves once at shared channels and joining them there. This satisfies Definition 6.10 by construction and costs a factor logarithmic in the number of allocated names, the depth of the chains. We state the discipline rather than this scheme because a cheaper scheme satisfying Definition 6.10 would serve equally; but an implementation that ignores the question is wrong, not merely unproved.

Remark 6.13 (Per-node channels). Within a single instance the allocation gadget’s internal channels must be per-node: a single shared u or z lets the tree mis-address its own leaves. Likewise, one spare leaf suffices for a self-instantiating unit such as Example 6.9, but a unit with nested inputs needs one per nested unit.

6.5 Freshness

Distinctness of leaves is not enough: they must also avoid the names of the translated program. This does not follow from Lemma 6.7, and it cannot be obtained by choosing a shape outside the constructor family, since by Definition 3.8 there is a member at every shape.

Lemma 6.14 (Phase-two freshness). *Let σ be an $\equiv_{\mathcal{N}}$ -normal name larger than every $\equiv_{\mathcal{N}}$ -normal name occurring in $\llbracket P \rrbracket$. Then no leaf beneath σ occurs in $\llbracket P \rrbracket$, and no name constructed at run time by a link of kind (o5) is a leaf of σ .*

Proof. L and R strictly increase size on normal forms, so every leaf exceeds σ and hence every name of $\llbracket P \rrbracket$; this gives the first claim. For the second, every leaf mentions σ hereditarily, while a link of kind (o5) builds its result from names received on source channels. Addresses never enter the payload of a source message — that is the “values may not be permuted” half of §6.4 read in the other direction — so no constructed name mentions σ . $\square$

The second clause is an obligation on the translation rather than a property of the calculus, and it is mechanically checkable: no address-valued channel may feed a constructor chain of kind (o5).

Remark 6.15 (Finiteness of the family, made statable). Lemma 6.14 also settles a question left open by Definition 3.8. ERECT erects only phase-one atoms, so the shapes a given program needs are those of atoms mentioning one of its bound names. The family therefore terminates at one level for each program, rather than regressing through constructors of constructors.

7 Correctness

Lemma 7.1 (Compositionality). $\llbracket C[P] \rrbracket = \llbracket C \rrbracket[\llbracket P \rrbracket]$, *on the nose*.

Proof. Induction on C . Each clause of Definition 6.3 mentions its sub-translations only under a seed abstraction, and the hole occurs in exactly one. $\square$

Lemma 7.2 (Linearity, per instance). *Within one instance of $\llbracket P \rrbracket^\sharp$, every allocated name is the subject of at most one atom and carries at most one message, and every erected atom participates in at most one reduction.*

Proof. Allocated names are leaves beneath the instance’s address, distinct by Lemma 6.7; each clause of Definition 6.3 emits each bound name in one subject position and at most one value position; every rule consumes both participants.

The claim is stated per instance, and that restriction is essential rather than cosmetic. Copies of compiled code arise as payloads, and a payload is instantiated verbatim, so a positional allocation would give every instance the same names — which is the failure of §6.1. What makes the lemma true is that the apparatus of an instance is allocated beneath an address the instance receives, and distinct instances hold addresses neither of which is a prefix of the other. $\square$

Definition 7.3 (Link expansion). $T' \succeq T$ if T' is obtained from T by replacing the subject a of finitely many atoms by distinct fresh proxies c and adjoining $\text{fw}(c, a)$ for a value-producing subject, $\text{fw}(a, c)$ for a value-consuming one.

Lemma 7.4. *If $T' \succeq T$ and T satisfies Lemma 7.2 then $T' \approx_{\llbracket \mathcal{C}_{\text{rho}} \rrbracket} T$.*

Proof. Relate T' to T when T' is such an expansion with some adjoined forwarders already discharged. A labelled step of T through an expanded atom is matched in T' by the relay step followed by the same rule, giving an expansion of the residue; a step of T' is either a relay, matched by the empty path, or an ordinary step matched directly. Linearity guarantees one one-shot forwarder per occurrence suffices, the atom firing at most once. No adjoined forwarder is a message, so no label distinguishes them. $\square$

Lemma 7.5 (Inertness). $\llbracket \text{for}(y \leftarrow x)P \rrbracket^\sharp$, once erected, has no reduction and contains no message.

Proof. The atoms present are the duplicators of D , the gate, and the links. None is a message, and the atoms of B are stored inside $\mathbf{q}$, hence are not subterms of any redex. No forwarder is present, so $\mathbf{q}(b, B)$ is inert. $\square$

Lemma 7.6 (Name usage). *Let y be bound in $\llbracket P \rrbracket$ by the translation of a rho input. Every occurrence of the corresponding proxy is the subject of an atom of $\llbracket P \rrbracket$ or an argument of a link. In particular an encoded context neither names the allocated apparatus of a term it is composed with, nor installs a forwarder at such a name.*

Proof. Induction on P using Definition 6.3: proxies are introduced only by Table 1 and used only there. For the second claim, $\llbracket C \rrbracket$ allocates beneath its own address, and by Lemma 6.7 those leaves are distinct from the leaves of the term plugged into the hole, the two addresses being prefix-incomparable. $\square$

Lemma 7.7 (Release). $\llbracket \text{for}(y \leftarrow x)P \rrbracket_s \mid \mathbf{m}(\llbracket x \rrbracket, @\llbracket R \rrbracket) \longrightarrow^* T$ with $T \approx_{\llbracket \mathcal{C}_{\text{rho}} \rrbracket} \llbracket P\{\text{@}R/y\} \rrbracket_s$, in $k + 3$ steps followed by at most k administrative steps, and no other sequence is available up to the order of independent steps.

Proof. By Lemma 6.2 the distributor delivers a copy to each p_i and the gate releases B . Each remaining copy meets its link. For (o1) and (o2) the link yields a one-shot forwarder, which is the corresponding link expansion; for (o3) it yields the message the translated output would have been; for (o4) it yields $\mathbf{m}(c_i, @\llbracket R \rrbracket)$, which with the $\mathbf{e}(c_i)$ in B gives $\llbracket R \rrbracket$, and $(*y)\{\text{@}R/y\} = R$ by Definition 2.3, so the occurrence is discharged exactly; for (o5) the chain of Proposition 3.12 yields $\mathbf{m}(c_i, @\llbracket S\{\text{@}R/y\} \rrbracket)$, node by node, since $\llbracket - \rrbracket$ is a homomorphism and each constructor reproduces one node of $\llbracket S \rrbracket$ with the received name in place of the occurrence. Hence $T \succeq \llbracket P\{\text{@}R/y\} \rrbracket_s$ and Lemma 7.4 applies. Uniqueness follows from Lemmas 7.5 and 7.2. $\square$

The (o4) case is where the source's substitution and the target's $\mathbf{e}$ rule coincide: under presentation B both deliver $\llbracket R \rrbracket$ directly, and under A both deliver $*\text{@}\llbracket R \rrbracket$, which the equation makes equal to it. The (o5) case is where the constructor earns its place.

Lemma 7.8 (Address independence). $\llbracket P \rrbracket^\sharp$ run at σ and at τ are $\llbracket \mathcal{C}_{\text{rho}} \rrbracket$ -bisimilar, for prefix-incomparable σ, τ .

Proof. Let $\beta_{\sigma \rightarrow \tau}$ send $\sigma \cdot w \mapsto \tau \cdot w$ and fix all other names; it is a bijection by Lemma 6.7 and extends to terms by metalevel replacement, descending into quotations. It is not a rho substitution and no combinator context performs it. Take $\mathcal{R} = \{(T, \beta_{s \rightarrow t}(T))\}$ for T reachable from $\llbracket P \rrbracket^\sharp$ at σ . A bijection on names preserves and reflects $\equiv_{\mathcal{N}}$, hence the applicability of every rule of Definition 3.3, including $\mathbf{e}$ and $\mathbf{cons}$, whose right-hand sides are computed from the redex and so commute with β . For the labels, Lemma 7.6 shows an encoded context never names an allocated leaf of the term it observes, so no label distinguishes T from its image. Lemma 6.11 is the same observation at run time: a permutation of addresses among instances is invisible, because only addresses ever cross a shared channel. $\square$

Theorem 7.9 (Full abstraction relative to encoded contexts). *For all rho processes P, Q ,*

$$P \approx_{\mathcal{C}_{\text{rho}}} Q \iff \llbracket P \rrbracket \approx_{\llbracket \mathcal{C}_{\text{rho}} \rrbracket} \llbracket Q \rrbracket.$$

Proof. Forward. Let $\mathcal{S}$ relate T to U whenever T and U are reachable from $\llbracket P' \rrbracket$ and $\llbracket Q' \rrbracket$ respectively, for some $P' \approx Q'$, by the sequences of Lemma 7.7, modulo link expansion and β . It is symmetric and contains $(\llbracket P \rrbracket, \llbracket Q \rrbracket)$. Given a label $\llbracket C \rrbracket$, Lemma 7.1 gives $\llbracket C \rrbracket \llbracket P' \rrbracket = \llbracket C[P'] \rrbracket$, so a target step from the plugged term is, by Lemma 7.5, either administrative — matched by the empty path, staying in $\mathcal{S}$ by Lemmas 7.4 and 7.8 — or the first step of the sequence of Lemma 7.7, in which case $C[P'] \rightarrow P''$ in the source and the matching source step for Q' transfers back along the same lemma.

Backward. A source step $C[P] \rightarrow P''$ is, by Lemma 7.7 read left to right, matched by $\llbracket C \rrbracket \llbracket P \rrbracket \rightarrow^* T \approx \llbracket P'' \rrbracket$; so a distinguishing source context yields a distinguishing encoded context, and the contrapositive is the claim. $\square$

Proposition 7.10 (The restriction is forced). *There are rho processes $P \approx_{\mathcal{C}_{\text{rho}}} Q$ and a combinator context $C \notin \llbracket \mathcal{C}_{\text{rho}} \rrbracket$ with $C[\llbracket P \rrbracket] \not\approx C[\llbracket Q \rrbracket]$.*

Proof. The seed is computed by the translation from the source, so an arbitrary combinator context can name s^w ; take $C = [\cdot] \mid \text{fw}(b, u)$ for the gate name b of a translated input and an observable u . By Lemma 6.2 this extracts the stored continuation at u , distinguishing two source terms that differ only in a continuation not otherwise reachable. By Lemma 7.6 no encoded context installs such a forwarder. $\square$

Remark 7.11. This is the same shape as the corresponding result in [4], and for the same underlying reason: the target manufactures names the source does not have, and an unrestricted observer of the target may use them. There the names are addresses; here they are seed powers. In neither case is the restriction a weakening of the theorem — it is the theorem’s shape. Note also that $\mathbf{q}$ is doing real work in Proposition 7.10: with the continuation stored as $\mathbf{m}(b, @B)$ the context would not even need to act, since the store would be a message and hence directly observable.

Remark 7.12 (Loud and quiet). With the constructors in the calculus the point generalises. A construction chain parks each intermediate result as a message on a constructed name, so the whole dataflow of name-building is observable to an unrestricted context, and Table 1 (o5) makes such chains common. The question $\mathbf{q}$ answers for the nullary case — should a stored value be loud or quiet? — therefore applies at every arity, and a uniform treatment would give each constructor a quiet variant emitting $\mathbf{q}(c, \cdot)$ rather than $\mathbf{m}(c, \cdot)$. We have not done this, and it is the main design question left open by §10.

8 Complexity of the target term

Let $|P|$ count atoms and $d(P)$ the nesting depth of input prefixes.

Proposition 8.1. $|\llbracket P \rrbracket^\sharp| = O(|P|)$.

Proof. The clauses for $\mathbf{0}$, parallel composition and output add at most one atom each. An input adds k duplicators, three gate atoms and k links, where k counts occurrences of the bound name; the body is translated once and appears once, inside a quotation. Summing k over binders counts each occurrence once. A link of kind (o5) is a chain of $O(|S|)$ constructors by Proposition 3.12, within the same bound since S is a subterm.

Phase two multiplies by a constant. Each bound name costs one allocation gadget of five atoms and three steps (Lemma 6.7), and each atom mentioning a bound name costs one constructor and one $\mathbf{e}$ (Remark 3.11); both counts are proportional to the phase-one output. Satisfying

Definition 6.10 by the scheme of Remark 6.12 multiplies by a further factor logarithmic in the number of allocated names. The run-time cost per instance rises likewise, which is the price of per-instance allocation and what a positional scheme was trying, unsoundly, to avoid.

One consequence is practical. Every loop descends one address level per unfolding, so names grow along any non-terminating computation; name growth is the common case rather than a pathology, and prefix sharing in an implementation's intern table is load-bearing for step cost rather than an optimisation. $\square$

Yoshida must distribute the prefix through the body: her rule (I) splits at every parallel composition, inserting a duplicator, and her rules (V) and (VI) delay every atom behind its own synchroniser. Each prefix therefore multiplies the body.

Proposition 8.2. *Let $P_d = \text{for}(x_1 \leftarrow a)\text{for}(x_2 \leftarrow a) \cdots \text{for}(x_d \leftarrow a)\mathbf{0}$, so $|P_d| = d + 1$ and $d(P_d) = d$. Yoshida's elimination yields $T(d) = (3^{d-1} + 1)/2$ atoms, where $T(1) = 1$ and $T(j + 1) = 3T(j) - 1$, while $||P_d|| = 3d + 1$.*

Proof. The innermost prefix gives one atom by her rule (III). If the body of the next prefix has n atoms, none mentioning the bound name, rule (I) inserts $n - 1$ duplicators along a binary tree and rule (VI) replaces each atom by a synchroniser and a relabelled copy, giving $2n + (n - 1)$. No other rule applies, so the count is exact. For the reflective encoding each prefix has no occurrences, contributing an empty distributor and one gate. $\square$

Corollary 8.3. *Prefix elimination with reflection is exponentially more succinct than without it, in the nesting depth of inputs.*

Remark 8.4. The mechanism is that Yoshida cannot store a continuation, so “wait, then behave like T ” must be implemented by rewriting T itself, and a constant-factor expansion composes multiplicatively down a nesting chain. Reflection stores T as the name $@T$ and pays only for the gate. We do not claim 3^d is optimal for her calculus, only that it is what her rules give; an unconditional lower bound would be a statement about all encodings.

9 An expressiveness lattice

Yoshida's separation results are not a single theorem but a lattice: subsets of the combinators ordered by inclusion, with each strict edge witnessed by a behaviour the smaller set cannot produce. This section sets up the corresponding lattice for the rho combinators. It has a feature hers does not, which is visible from the start and is a consequence of having no name hiding: some edges are strict against arbitrary observers and collapse against encoded ones, so what we have is not one lattice but a family indexed by the observer class of Definition 5.1.

Throughout, $\mathbf{m}$ is not a candidate for removal: it is the medium, and a calculus without it has no communication to analyse. The candidates are the six routers $\mathbf{d}, \mathbf{k}, \mathbf{fw}, \mathbf{b}_l, \mathbf{b}_r, \mathbf{s}$, the destructor $\mathbf{e}$, the store $\mathbf{q}$, and the four constructors.

9.1 Three invariants

Each proved edge below comes from an invariant preserved by every rule of the smaller set and violated by a witness in the larger.

Lemma 9.1 (Name closure). *Let S be a set of constructors and T a term over S . Every name occurring in a reduct of T lies in the closure of $\mathcal{N}^*(T)$ under the node shapes $\{A : \text{cons}_A \in S\}$, modulo $\equiv_{\mathcal{N}}$.*

Proof. Induction on reduction. Every rule other than the constructor rules has, on its right, only names occurring on its left: for $e(a) \mid m(a, @P) \rightarrow P$ the names of P are hereditarily among those of $@P$, and for $fw(a, b) \mid q(a, P) \rightarrow m(b, @P)$ both b and $@P$ occur in the redex. A constructor rule for shape A emits $m(c, @A(v_1, \dots, v_n))$, whose name is A applied to names already present. Structural congruence introduces no names. $\square$

Lemma 9.2 (Weighted atom count). *Weight b_l , b_r and s at 2 and every other non-message atom at 1. Every rule but that for e strictly decreases the total weight.*

Proof. d , k , fw and each constructor consume a weight-1 atom and produce none, so $1 \rightarrow 0$. The three binders consume themselves at weight 2 and produce fw at weight 1, so $2 \rightarrow 1$. The rule for q consumes fw and a q , so $2 \rightarrow 0$.

The unweighted count does not work, and the reason is recorded in the rule itself: $b_l(a, b) \mid m(a, v) \rightarrow fw(v, b)$ consumes one non-message atom and produces one, so the plain count is preserved rather than decreased. Weighting the binders above fw is the minimal repair. $\square$

Lemma 9.3 (Message count). *d is the only rule that increases the number of messages while consuming no q and opening no payload.*

Proof. By inspection: d takes one message to two; fw and each constructor are non-increasing; k , b_r , b_l , s decrease; q adds one but consumes a q ; e replaces a message by its payload. $\square$

9.2 The constructors: a Boolean sublattice

Definition 9.4. For a node shape A of arity n , let Spec_A be the behaviour: on receiving a name v at a , emit $m(A(v, k_2, \dots, k_n), @0)$ for fixed static k_i .

Proposition 9.5. *Spec_A is realised in any calculus containing cons_A , d , b_r and m , and in no calculus whose constructor set omits cons_A , relative to any observer class containing the contexts that supply a value at a and listen at $A(v, \vec{k})$.*

Proof. Realisation: $\text{cons}_A(p_1, \dots, p_n, c)$ with the received value delivered at p_1 and $m(p_i, k_i)$ for $i \geq 2$ yields $m(c, @A(v, \vec{k}))$; then $b_r(c, c')$ turns that into $fw(c', @A(v, \vec{k}))$, which relays $m(c', @0)$ to its destination. For the converse, let T omit cons_A . $\mathcal{N}^*(T)$ is finite, so choose v with $A(v, \vec{k}) \notin \mathcal{N}^*(T) \cup \mathcal{N}^*(v)$; by Lemma 9.1 the closure of $\mathcal{N}^*(T) \cup \mathcal{N}^*(v)$ under the remaining shapes contains no name with root A that is not already there, so no message is ever emitted at $A(v, \vec{k})$ and the observer's label is not matched. $\square$

Corollary 9.6. *The four constructors are pairwise independent, so the constructor axis is a Boolean lattice on four generators. The witnesses are the rho terms*

$$\begin{aligned} \text{cons}_l : & \quad \text{for}(y \leftarrow a) @(*y \mid *y)! (0) \\ \text{cons}_m : & \quad \text{for}(y \leftarrow a) @ (y! (0))! (0) \\ \text{cons}_d, \text{cons}_s : & \quad \text{for}(y \leftarrow a) @ (\text{for}(z \leftarrow y) T)! (0) \end{aligned}$$

the last with z occurring in T for cons_d and not for cons_s , by Proposition 3.12.

Remark 9.7. The two lower witnesses need only their own constructor; the two upper ones need cons_l as well, since the image of an input is a parallel composition of a head atom with static material. So the axis is a Boolean lattice on $\{\text{cons}_m, \text{cons}_d, \text{cons}_s\}$ over a base containing cons_l , with cons_l itself the bottom step. The bottom edge is the statement that the published calculus cannot encode the rho calculus at all: it can quote code, but it cannot compute names.

Remark 9.8. There is an instructive mirror image in [2], used in [4]: the rho calculus manufactures new free — hence observable and substitutable — names at runtime, which is why no encoding of the π -calculus into rho is fully abstract against unrestricted rho observers. The same capability read from the other side is the bottom edge of this axis. One calculus has too much name construction for π to be safe in it; the other, without constructors, has too little for rho to fit.

9.3 The destructor

Proposition 9.9. *Without e , every reduction sequence from T has length at most the weighted atom count of Lemma 9.2, hence at most twice the number of non-message atoms of T . So the calculus is strongly normalising, hence not Turing complete, and e is not definable in it.*

Proof. Lemma 9.2, the rule for e being the only one whose right side is an arbitrary process; the weight of a term is at most twice its non-message atom count. $\square$

The destructor therefore carries the computational power that reflection adds, and the recursion construction of §6 depends on it. The constructors do not restore it: they build names, and without e no name is ever opened. This edge is independent of the constructor axis, so the two together already give a square.

9.4 Duplication

Conjecture 9.10. *Without d , the calculus is strongly normalising even in the presence of e and all four constructors.*

The evidence is Lemma 9.3 together with the observation that e can only unfold a payload, and a payload is either written into the term or built by constructors from names already present. Self-reference in the rho calculus requires the received name twice — this is why the recursion combinator is $\text{for}(y \leftarrow x)(x!(*y) \mid *y)$ — and without d a message cannot be duplicated. What blocks a proof is that constructors can build names of unbounded size, so the usual size measure does not descend; a divergent term, if one exists, would have to grow its own code through a constructor chain rather than by duplication. Settling this decides whether divergence is a property of e alone or jointly of e and d , and it is the most interesting open edge in the lattice.

9.5 Routers, storage, and the observer class

Two edges here are not about expressive power in the usual sense, and they are where the lattice becomes observer-relative.

Proposition 9.11 (fw is redundant exactly when q is absent). *In a calculus without q , $\text{fw}(a, b) \approx_{\mathcal{D}} d(a, b, c) \mid k(c)$ for c not otherwise used and any $\mathcal{D}$ not naming c . In a calculus with q , no such equation holds: the rule for q requires an fw atom, so without fw in the atom set every q is inert and no gate ever fires.*

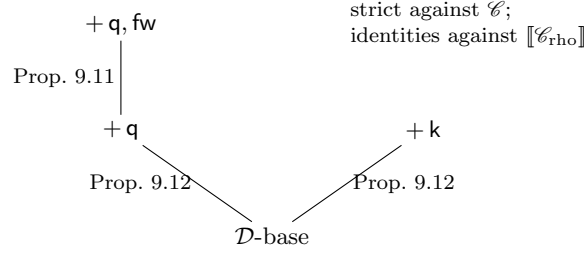


Figure 1: The observational lattice over the same base. Each edge is strict against arbitrary combinator contexts and an identity against encoded ones, so this entire figure collapses to its bottom point in Figure 2. The fw edge sits above the q node rather than over the base, since fw is redundant when q is absent and essential when it is present. No edge is drawn from the base to fw alone: by Proposition 9.11 there is none.

Proof. For the first, $\text{d}(a, b, c) \mid \text{k}(c) \mid \text{m}(a, v) \longrightarrow \text{m}(b, v) \mid \text{m}(c, v) \mid \text{k}(c) \longrightarrow \text{m}(b, v)$, two steps against one, with no intermediate barb at a name $\mathcal{D}$ can see. The atoms b_r , b_l and s would then be read as producing $\text{d} \mid \text{k}$ in place of fw . The second is immediate from Definition 3.3. $\square$

Proposition 9.12 (Collapsing edges). *Relative to $\mathcal{D} = \llbracket \mathcal{C}_{\text{rho}} \rrbracket$: $\text{q}(a, P)$ and $\text{m}(a, @P)$ are interchangeable, and $\text{k}(a)$ and $\text{s}(a, b, c)$ for constructed b, c are interchangeable. Relative to arbitrary combinator contexts, neither is.*

Proof. q and m have the same rule against fw and differ only in being reachable by the other consumers and in carrying a barb; by Lemma 7.6 an encoded context neither names a constructed a nor installs a consumer there. $\text{k}(a)$ and $\text{s}(a, b, c)$ differ by the residue $\text{fw}(b, c)$, inert when nothing arrives at b . For the converse in each case, Proposition 7.10 supplies the distinguishing context. $\square$

So q occupies the same point as m on the expressiveness ordering and a strictly different one on the observational ordering. These are two orderings on the same subsets and they do not coincide — Figures 2 and 1 — which is the structural difference from Yoshida’s setting: she has new , so a constructed name can be hidden and the two orderings agree. Several of her separation arguments turn on exactly that hiding and do not transfer; they want redoing rather than citing, and we have not done so here.

9.6 Summary

edge	status	by
cons_l over no constructors	strict	Prop. 9.5
cons_m , cons_d , cons_s , pairwise	strict	Cor. 9.6
e over no e	strict	Prop. 9.9
d over no d	conjectured strict	Conj. 9.10
fw , given q	strict	Prop. 9.11
fw , without q	collapses, given d, k	Prop. 9.11
q over m	collapses for encoded observers	Prop. 9.12
k over s	collapses for encoded observers	Prop. 9.12
b_l , b_r , s , pairwise	open	

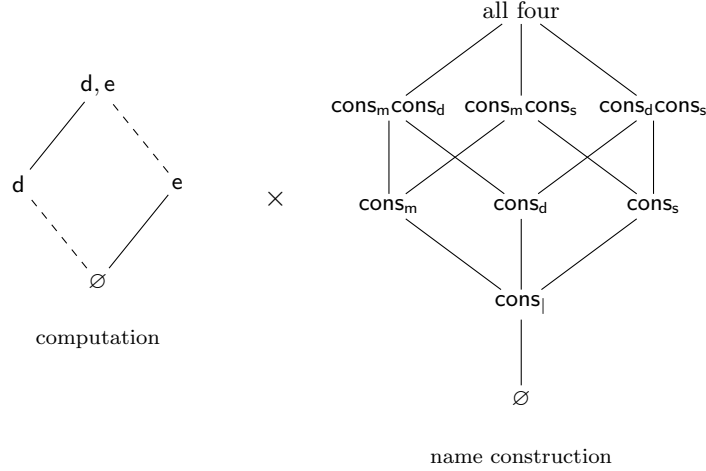


Figure 2: The expressive lattice, as a product of two independent axes over the base of $\mathbf{m}$ and the routers. Nodes above cons_l carry it implicitly. Solid edges are strict by Propositions 9.5 and 9.9 and Corollary 9.6; dashed edges are Conjecture 9.10 and the open question of whether duplication alone adds anything in the absence of $\mathbf{e}$. The atoms $\mathbf{q}$, $\mathbf{k}$ and $\mathbf{fw}$ do not appear: on this ordering $\mathbf{q}$ sits at the point of $\mathbf{m}$, and $\mathbf{k}$ and $\mathbf{fw}$ are derivable from the remaining routers (Propositions 9.11 and 9.12).

The last row is the gap. Yoshida separates her binders from one another, and the corresponding question here is genuinely open rather than inherited, because her arguments use hiding and ours cannot.

10 An alternative presentation: constructors only

The calculus of §3 has three kinds of atom doing three jobs: routers ($\mathbf{d}, \mathbf{k}, \mathbf{fw}, \mathbf{b}_l, \mathbf{b}_r, \mathbf{s}$), which move names; a destructor ($\mathbf{e}$), which opens one; and value-bearing atoms ($\mathbf{m}, \mathbf{q}, \text{cons}$). The third group is not three things but one.

Remark 10.1. Read a constructor as a dataflow node: $\text{cons}_A(a_1, \dots, a_n, c)$ says that when names are available at $a_1, \dots, a_n$, the name A of them is available at c . Then $\mathbf{m}(c, v)$ is the nullary case — the name v is available at c , unconditionally — and $\mathbf{q}(c, v)$ is the nullary case in a quiet variant. On this reading neither $\mathbf{m}$ nor $\mathbf{q}$ is primitive: the calculus consists of routers, one destructor, and a constructor family whose arity-zero members are the messages.

Two things recommend this presentation. It is uniform: every atom takes names and every rule is either routing, opening, or building, with no atom indexed by a process. And it makes Remark 7.12 a question about the family rather than about two special atoms — whether a constructor’s output is loud or quiet is a single design parameter, and $\mathbf{m}$ versus $\mathbf{q}$ is its nullary instance.

Two things count against it, and we record them rather than settle them. Barbs, or their context-labelled counterpart, need a notion of message; if messages are nullary constructors then observation must be defined on the constructor family, which is a mild complication but changes what Lemma 7.6 has to say. The second objection has been overtaken. A calculus closed under its own constructions needs one member per atom shape, including members for

the constructors themselves, since a constructed name may quote a term containing one — and §6.3 makes that closure compulsory rather than fastidious, since erecting an allocated atom calls for a constructor at its shape. The family of Definition 3.8 is thus the working signature, and the constructors-only reading of this section describes the calculus as it now stands rather than a variant of it.

11 Corrigenda to [5]

Restricted to what bears on the rho calculus; the material specific to the π -calculus route is out of scope here.

1. **The drop and the guard are conflated.** The published grammar writes both as $*$ and lists the guard under the process production, which makes every running process a redex (Proposition 3.5). They must be separate constructors. The equation $*(@P) = P$ may then be kept or omitted (§3.3); it is not the source of the unsoundness.
2. **The binders produce messages rather than forwarders** in some statements of the rules, which collapses them into *forward*; see Remark 3.4, which also records the arity slips in *fw* and in the constructors' delivery channel.
3. **No rho-to-combinator translation is given.** The outline promises that the encoding of a source calculus into the combinators composes with an encoding into the rho calculus, but the translation defined in the body has Yoshida terms as its domain. §6 supplies the missing map, and §9 shows that no such map exists without a name constructor.
4. **No prefix-elimination lemma.** The draft states two theorems and proves no lemmas. The prefixed form $\text{for}(x \leftarrow p)C$ is never given a semantics in the combinator calculus, so the correctness of elimination is neither stated nor proved. Lemmas 6.2 to 7.7 are the analogue here.
5. $\equiv_{\mathcal{N}}$ **is undefined for the combinators** yet is used in the side condition on observation.
6. **Apparatus names are positional.** The draft's constructed names are functions of syntactic position, which is unsound for code instantiated more than once (§6.1); they also quote the subterm they must be fresh for, making the output quadratic before other costs. Allocation beneath a run-time address (§6.3) fixes both.
7. **Typing slips.** $x^l := @b_l(x, m(x, *x))$ puts a process in the second argument of b_l , which takes names. The derivation following the definition of the replication server carries a spurious extra copy of $m(x, @(D \mid P))$ on its third line; the calculation is otherwise correct. Polarity annotations appear inside quoted terms, and several where-clauses construct names mentioning a bound variable, which is a metavariable rather than a name of the calculus.

12 Open questions

1. **Minimal labels.** Theorem 7.9 uses enabling contexts. The minimal-label form, via relative pushouts taken in the image of the translation, is the discipline of [1]; the forward half descends for free and the backward half is what is owed.

2. **Loud and quiet constructors** (Remark 7.12). Should every constructor have a quiet variant, and does the quiet calculus admit a sharper form of Proposition 7.10?
3. **Name growth.** With the constructors, names are no longer subterms of the program: they grow at runtime. The consequences for the decidability of $\equiv_{\mathcal{N}}$ in practice, and for cost accounting, want examining.
4. **The open edges of the lattice** (§9): Conjecture 9.10, and the mutual independence of $\mathbf{b}_l$, $\mathbf{b}_r$ and $\mathbf{s}$ without recourse to hiding.
5. **How large must the constructor family be?** Proposition 3.12 gives four for static apparatus and Remark 3.13 shows phase two needs one per atom shape. Is the full closure necessary, or does some arrangement of the allocation erect apparatus with fewer? And does reflection make any of Yoshida’s six routers redundant?

References

- [1] C.B. Wells, M. Stay and L.G. Meredith. *Behavior in higher-order languages: admissible observers, relative pushouts, and the encoding of extended theories*. In preparation, F1R3FLY.io, 2026.
- [2] S. Lybech. *Encodability and separation for a reflective higher-order calculus*. EXPRESS/SOS 2022, EPTCS 368, 95–112.
- [3] L.G. Meredith and M. Radestock. *A reflective higher-order calculus*. Electronic Notes in Theoretical Computer Science 141(5):49–67, 2005.
- [4] L.G. Meredith. *Namespaces without a name server: freshness by communication, and a fully abstract encoding of the π -calculus in the rho calculus*. F1R3FLY.io, 2026.
- [5] L.G. Meredith and M. Stay. *Name-free combinators for concurrency*. Working draft, rho4u.
- [6] N. Yoshida. *Minimality and separation results on asynchronous mobile processes: representability theorems by concurrent combinators*. Theoretical Computer Science 274(1–2):231–276, 2002.